

Google Sheets API Task

Lectures review:

Google Sheets API in Python | Getting Started

Google Sheets API in Python | Creating Google Sheets Files

Google Sheets API in Python | Writing Data To Google Sheets

Google Sheets API in Python | Reading Data From Google Sheets

Google Sheets API in Python | Data Formatting Basics

Google Sheets API in Python | Append Rows and Columns

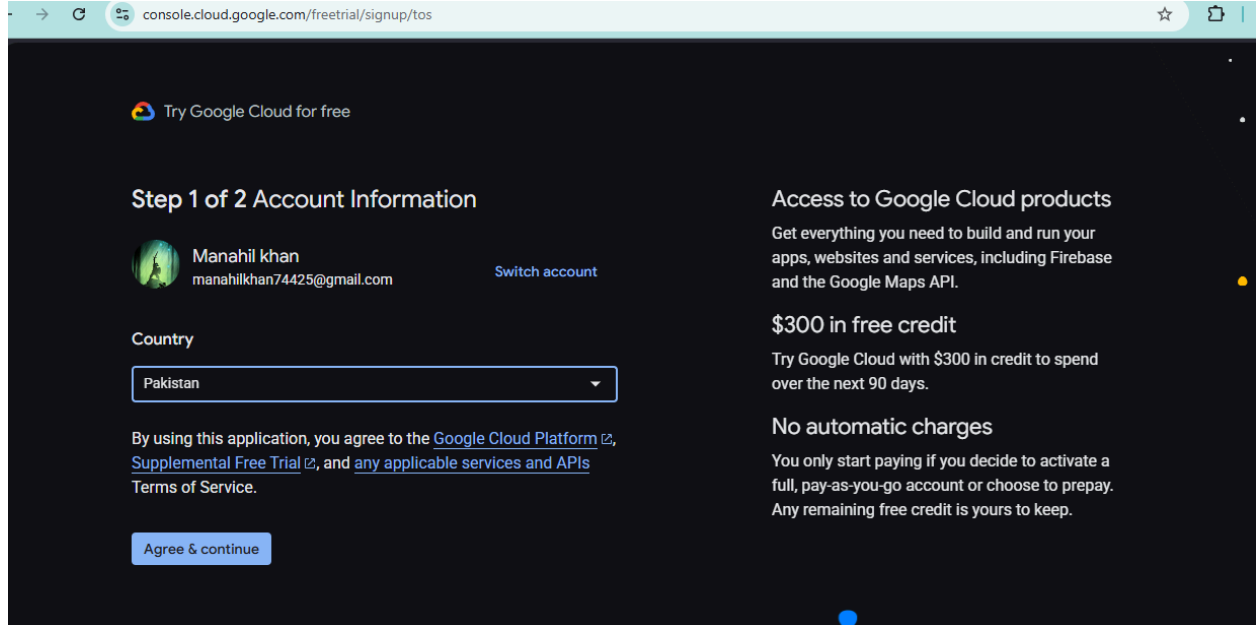
Google Sheets API in Python | Insert Rows and Columns

Setting up Python environment and Jupyter Notebook:

Requirements installed:

```
pip install google-api-python-client google-auth-httpplib2 google-auth-oauthlib pandas openpyxl
```

Creating Google cloud account:



The screenshot shows the 'Step 1 of 2 Account Information' page for creating a Google Cloud account. The user is Manahil Khan with email manahilkhan74425@gmail.com. The country is set to Pakistan. There is a 'Switch account' link. A blue 'Agree & continue' button is at the bottom left. On the right, there is information about access to Google Cloud products, \$300 in free credit, and no automatic charges.

Try Google Cloud for free

Step 1 of 2 Account Information

 **Manahil Khan**
manahilkhan74425@gmail.com [Switch account](#)

Country
Pakistan

By using this application, you agree to the [Google Cloud Platform](#), [Supplemental Free Trial](#), and [any applicable services and APIs Terms of Service](#).

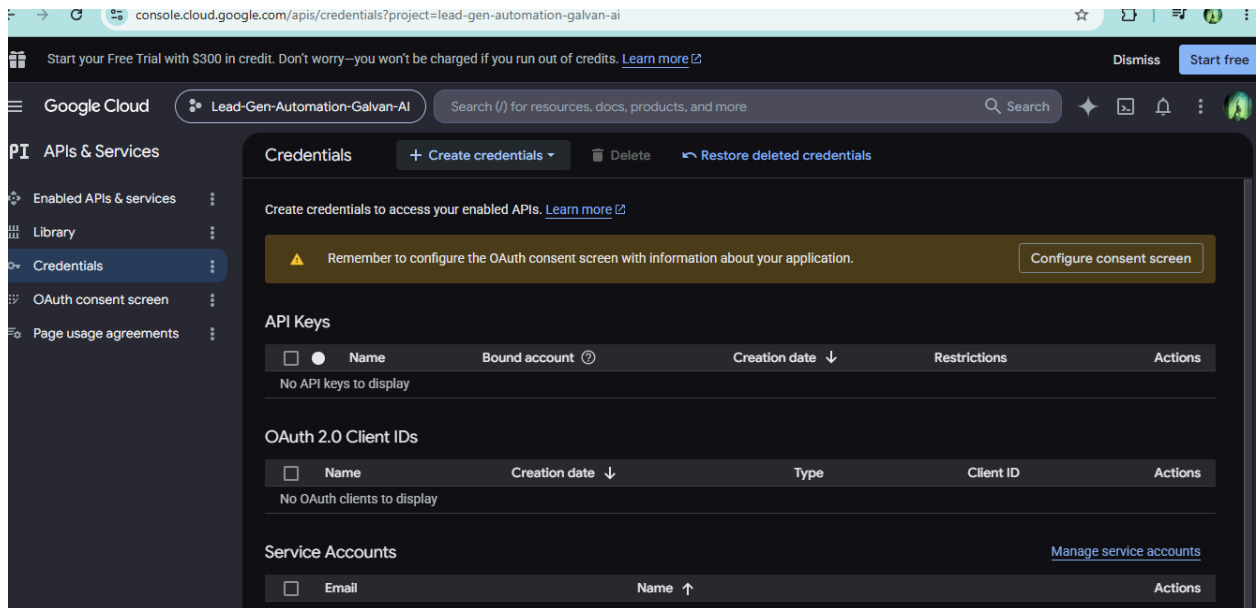
[Agree & continue](#)

Access to Google Cloud products
Get everything you need to build and run your apps, websites and services, including Firebase and the Google Maps API.

\$300 in free credit
Try Google Cloud with \$300 in credit to spend over the next 90 days.

No automatic charges
You only start paying if you decide to activate a full, pay-as-you-go account or choose to prepay. Any remaining free credit is yours to keep.

Enabling Sheets API & Creating Service Account:



The screenshot shows the 'Credentials' page in the Google Cloud Console for the project 'Lead-Gen-Automation-Galvan-AI'. The left sidebar shows the 'APIs & Services' menu with 'Credentials' selected. The main content area has a 'Create credentials' button and a warning to configure the OAuth consent screen. Below this are sections for 'API Keys', 'OAuth 2.0 Client IDs', and 'Service Accounts', all of which are currently empty.

Start your Free Trial with \$300 in credit. Don't worry—you won't be charged if you run out of credits. [Learn more](#)

Dismiss [Start free](#)

Google Cloud **Lead-Gen-Automation-Galvan-AI** Search (/) for resources, docs, products, and more

APIs & Services

- Enabled APIs & services
- Library
- Credentials**
- OAuth consent screen
- Page usage agreements

Credentials [+ Create credentials](#) [Delete](#) [Restore deleted credentials](#)

Create credentials to access your enabled APIs. [Learn more](#)

Remember to configure the OAuth consent screen with information about your application. [Configure consent screen](#)

API Keys

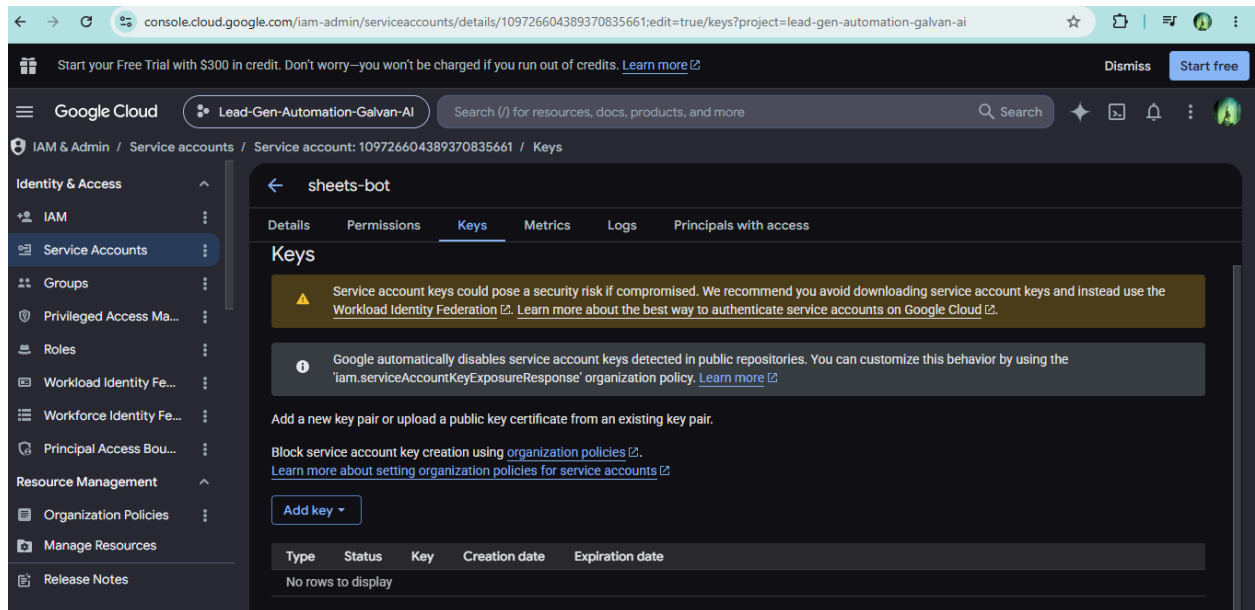
☐	Name	Bound account	Creation date	Restrictions	Actions
No API keys to display					

OAuth 2.0 Client IDs

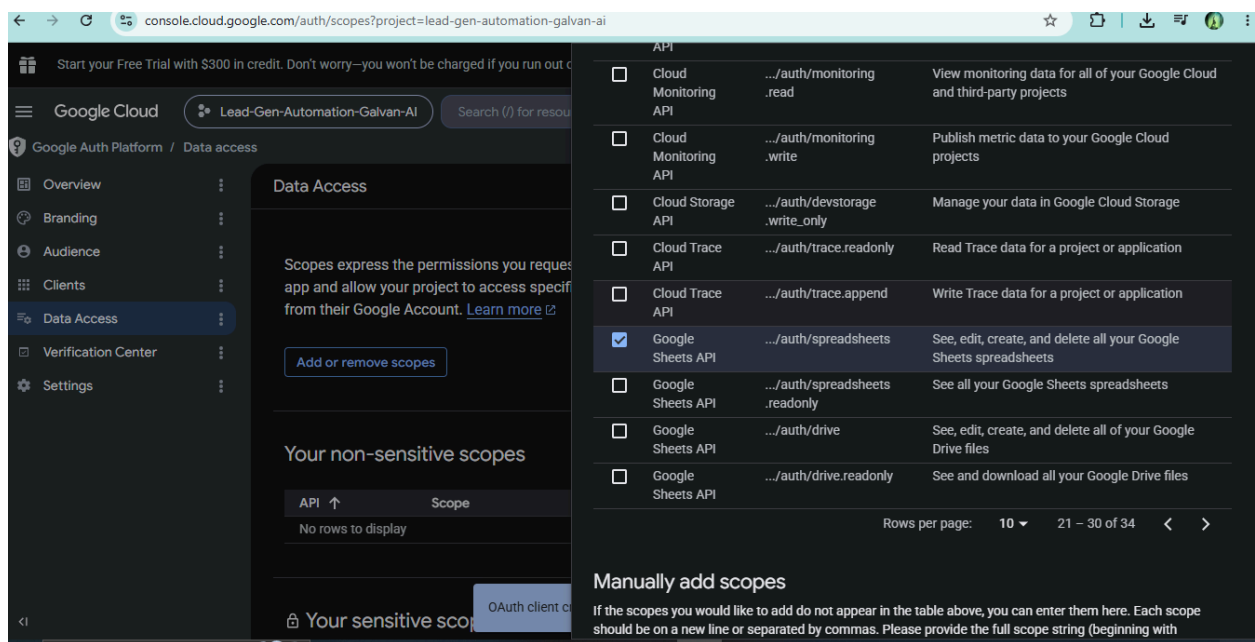
☐	Name	Creation date	Type	Client ID	Actions
No OAuth clients to display					

Service Accounts [Manage service accounts](#)

☐	Email	Name	Actions
No service accounts to display			



Service Account not enabling so shift to OAuth 2.0 token:

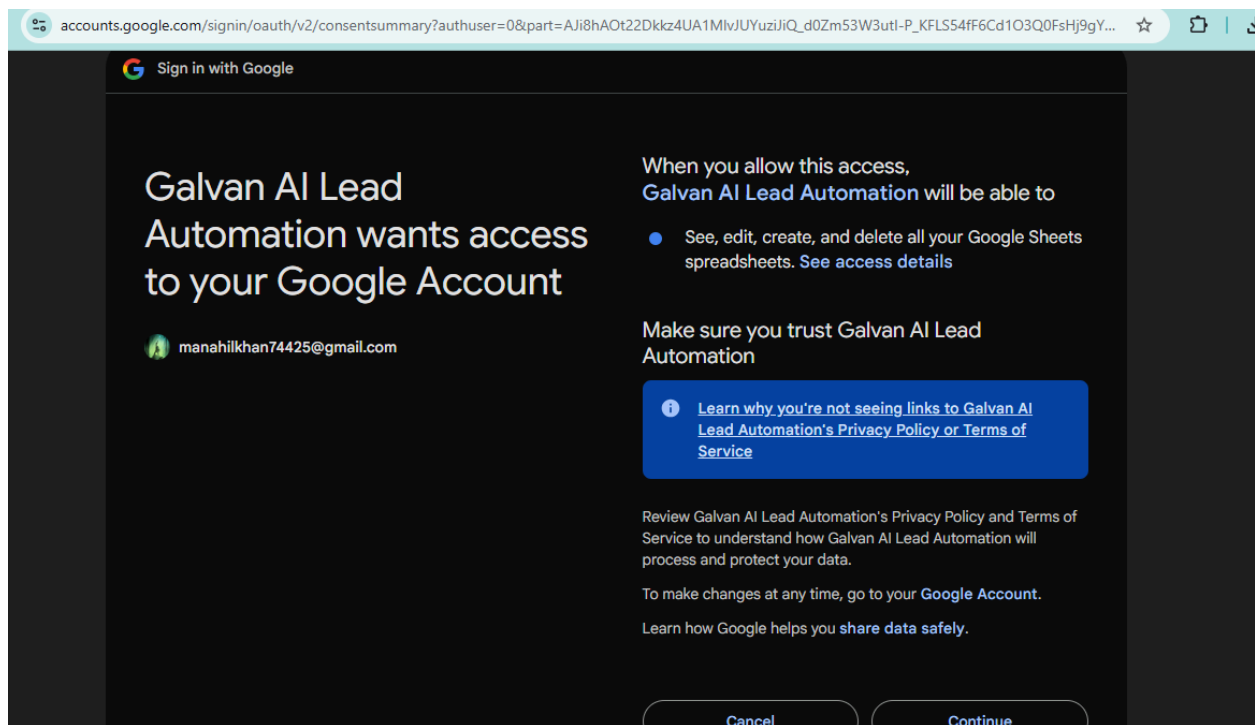


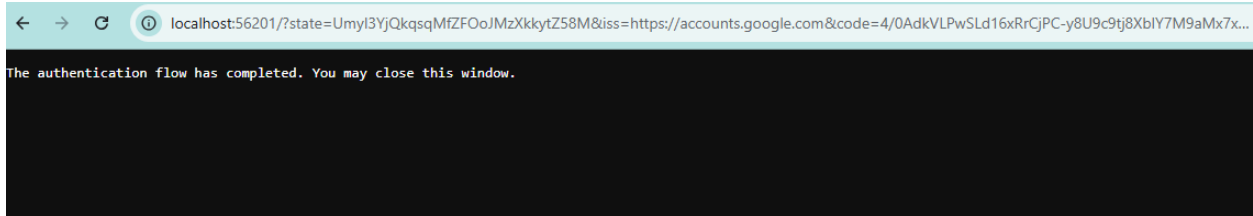
Running code for initial login access and Authentication:

```
import os
import pandas as pd
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from google.auth.transport.requests import Request
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError

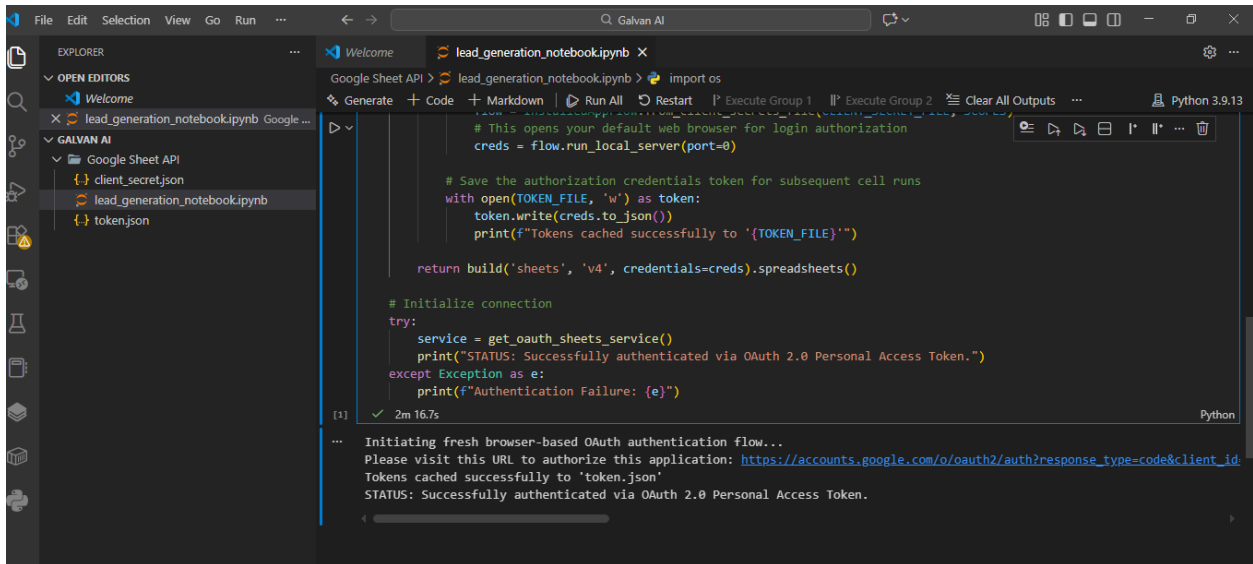
# Global Variables
SCOPES = ['https://www.googleapis.com/auth/spreadsheets']
CLIENT_SECRET_FILE = 'client_secret.json'
TOKEN_FILE = 'token.json'
SPREADSHEET_ID = "1btqCNcTbWSyF8ahVr2DT0cLVukne5nWTUwTLN9xePGU" # <-- Use any sheet YOU have access to
TARGET_RANGE = "Sheet1!A:F"

def get_oauth_sheets_service():
    """
    Handles OAuth2 user flow initialization, token caching,
    and returns an authorized Google Sheets API resource client.
    """
```

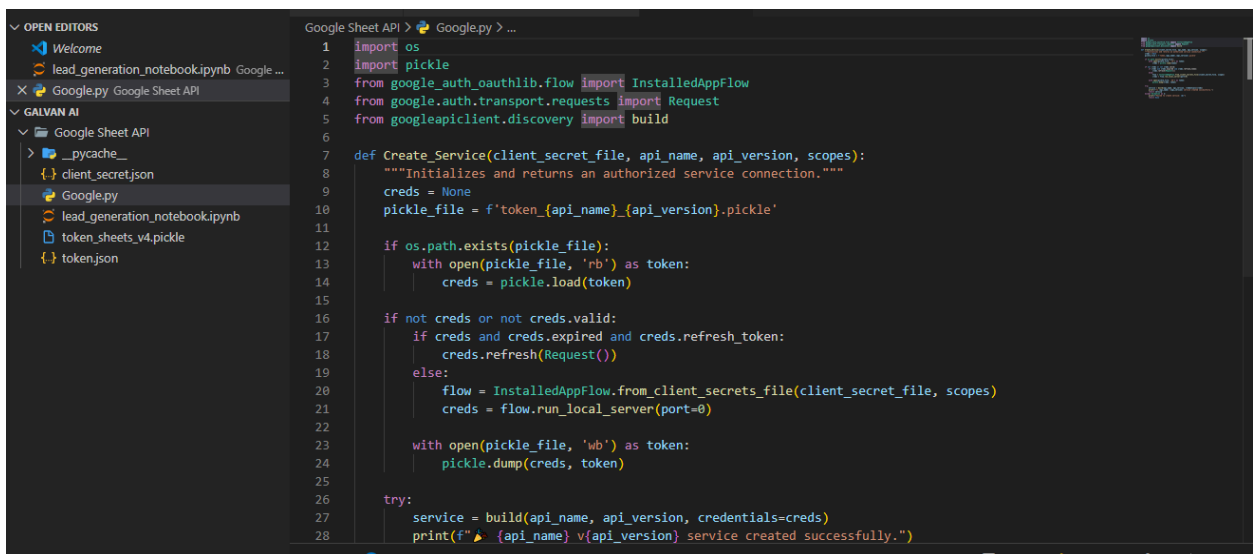




File Structure and Authorization token:



Separate API authorization and initialization file created ([Google.py](#)):



Perform basic functions of:

- Fetch data from Google sheets

Get:

```
Fetch

response = sheets.values().get(
    spreadsheetId=SPREADSHEET_ID,
    range="Sheet1!A1:F5"
).execute()

rows = response.get('values', [])
print("Data Fetched:")
for row in rows:
    print(row)
```

✓ 0.8s

Data Fetched:

- Update:

```
Update

# Let's set up column headers explicitly
headers_payload = {
    "values": ["Lead Name", "Email", "Phone", "Company", "Lead Source", "Status"]
}

sheets.values().update(
    spreadsheetId=SPREADSHEET_ID,
    range="Sheet1!A1:F1",
    valueInputOption="USER_ENTERED",
    body=headers_payload
).execute()
print("✓ Headers successfully forced into Row 1.")
```

[6] ✓ 0.7s

... ✓ Headers successfully forced into Row 1.

Untitled spreadsheet

File Edit View Insert Format Data Tools Gemini Extensions Help

100% | \$ % .0 .00 123 | Defaul... | - 10 + | B I ÷ A |

A1 *fx* Lead Name

	A	B	C	D	E	F	G	H
1	Lead Name	Email	Phone	Company	Lead Source	Status		
2								
3								
4								
5								
6								
7								
8								

- Store data into Google sheets

docs.google.com/spreadsheets/d/1btqCNcTbWSyF8ahVr2DTocLVukne5nWTUWTLN9xePGU/edit?gid=0#gid=0

Untitled spreadsheet

File Edit View Insert Format Data Tools Gemini Extensions Help

100% | \$ % .0 .00 123 | Defaul... | - 10 + | B I ÷ A |

E7 *fx*

	A	B	C	D	E	F	G
1	Lead Name	Email	Phone	Company	Lead Source	Status	
2	Natasha Romanoff	natasha@shield.gov	555-0300	S.H.I.E.L.D.	cold_outreach	new	
3	Clint Barton	hawkeye@avengers.org	555-0400	Avengers Academy	referral	contacted	
4	Wanda Maximoff	wanda@westview.io	555-0500	Independent	website	qualified	
5							
6							
7							
8							
9							

- Appending (adding more data):

Appending (Adding New Rows)

```
new_leads = {
  "values": [
    ["Tony Stark", "tony@stark.com", "555-0100", "Stark Industries", "linkedin", "new"],
    ["Bruce Banner", "hulk@avengers.org", "555-0200", "Gamma Labs", "website", "qualified"]
  ]
}

sheets.values().append(
  spreadsheetId=SPREADSHEET_ID,
  range="Sheet1!A:F",
  valueInputOption="USER_ENTERED",
  insertDataOption="INSERT_ROWS",
  body=new_leads
).execute()
print("✅ 2 New leads appended directly below existing records.")
```

✓ 1.5s

	A	B	C	D	E	F
1	Lead Name	Email	Phone	Company	Lead Source	Status
2	Natasha Romanoff	natasha@shield.gov	555-0300	S.H.I.E.L.D.	cold_outreach	new
3	Clint Barton	hawkeye@avengers.org	555-0400	Avengers Academy	referral	contacted
4	Wanda Maximoff	wanda@westview.io	555-0500	Independent	website	qualified
5	Tony Stark	tony@stark.com	555-0100	Stark Industries	linkedin	new
6	Bruce Banner	hulk@avengers.org	555-0200	Gamma Labs	website	qualified
7						
8						

- Batch Fetch (Read Multiple Separate Ranges at Once)

```
Batch Fetch (Read Multiple Separate Ranges at Once)
```

```
> batch_fetch_response = sheets.values().batchGet(
    spreadsheetId=SPREADSHEET_ID,
    ranges=["Sheet1!A1:B1", "Sheet1!E2:F3"]
).execute()

value_ranges = batch_fetch_response.get('valueRanges', [])
for vr in value_ranges:
    print(f"Range: {vr.get('range')} -> Values: {vr.get('values')}")
```

14] ✓ 4.1s

```
... Range: Sheet1!A1:B1 -> Values: [['Lead Name', 'Email']]
    Range: Sheet1!E2:F3 -> Values: [['cold_outreach', 'new'], ['referral', 'contacted']]
```

- Batch Update:

```
Batch Update (Write to Multiple Ranges at Once)
```

```
> batch_update_payload = {
    "valueInputOption": "USER_ENTERED",
    "data": [
        {"range": "Sheet1!F2", "values": [["contacted"]]}, # Update Tony's pipeline status
        {"range": "Sheet1!F3", "values": [["nurturing"]]} # Update Bruce's pipeline status
    ]
}

sheets.values().batchUpdate(
    spreadsheetId=SPREADSHEET_ID,
    body=batch_update_payload
).execute()
print("✅ Batch update operations processed across distinct ranges successfully.")
```

15] ✓ 0.9s

```
... ✅ Batch update operations processed across distinct ranges successfully.
```

Lead Generation spreadsheet ☆ 📁 ☁

File Edit View Insert Format Data Tools Gemini Extensions Help

🔍 ↶ ↷ 🖨 🗑 100% | \$ % .0 .00 123 | Defaul... | - 10 + | B I ⚡ A | 📄 🗃 📊 📈 📉 📉 📉

F6 | 📄 qualified

	A	B	C	D	E	F	G
1	Lead Name	Email	Phone	Company	Lead Source	Status	
2	Natasha Romanoff	natasha@shield.gov	555-0300	S.H.I.E.L.D.	cold_outreach	contacted	
3	Clint Barton	hawkeye@avengers.org	555-0400	Avengers Academy	referral	nurturing	
4	Wanda Maximoff	wanda@westview.io	555-0500	Independent	website	qualified	
5	Tony Stark	tony@stark.com	555-0100	Stark Industries	linkedin	new	
6	Bruce Banner	hulk@avengers.org	555-0200	Gamma Labs	website	qualified	
7							
8							

- Get or store data based on conditions
- Make sure you cover majority of the case scenarios
- Implement data validation
- Implement error handling